

//

explicit(bool)

Contents

- [Motivation](#)
- [Proposal](#)
- [Wording](#)
- [Acknowledgements](#)

Motivation

When writing a class template which wraps a member of template parameter type, it's useful to expose constructors that allow the user to construct the member in place. In order to do this properly, fulfilling the usage that the class designer expects, such constructors need to be `explicit` or not based on whether the corresponding class's constructor is `explicit` or not. Conditionally explicit constructors appear throughout the standard library in some of the most commonly used utility types - `std::pair`, `std::tuple`, and now `std::optional` and `std::variant` too, among others. This permits very natural code:

```
pair<string, string> safe() {
    return {"meow", "purrr"}; // ok
}

pair<vector<int>, vector<int>> unsafe() {
    return {11, 22}; // error
}
```

Despite being very useful functionality, it is quite cumbersome to implement. While conceptually we talk about a constructor being "conditionally `explicit`," the only way to implement that functionality is to write two constructors that are mutually disjoint - both of which beyond that do exactly the same thing. This gets a little better with concepts, where at least we don't have to duplicate the convertibility trait, but either way we have two duplicated constructors, and you just have to know why they're written the way they are. Here is an example from `std::pair`:

C++17 today (SFINAE)	With Concepts
<pre>template <typename T1, typename T2> struct pair {</pre>	<pre>template <typename T1, typename T2> struct pair {</pre>

```

template <typename U1=T1, typename U2=T2,
        std::enable_if_t<
            std::is_constructible_v<T1, U1> &&
            std::is_constructible_v<T2, U2> &&
            std::is_convertible_v<U1, T1> &&
            std::is_convertible_v<U2, T2>
        , int> = 0>
constexpr pair(U1&&, U2&& );

template <typename U1=T1, typename U2=T2,
        std::enable_if_t<
            std::is_constructible_v<T1, U1> &&
            std::is_constructible_v<T2, U2> &&
            !(std::is_convertible_v<U1, T1> &&
              std::is_convertible_v<U2, T2>)
        , int> = 0>
explicit constexpr pair(U1&&, U2&& );
};

```

```

template <typename U1=T1, typename U2=T2>
requires std::is_constructible_v<T1, U1> &&
         std::is_constructible_v<T2, U2> &&
         std::is_convertible_v<U1, T1> &&
         std::is_convertible_v<U2, T2>
constexpr pair(U1&&, U2&& );

template <typename U1=T1, typename U2=T2>
requires std::is_constructible_v<T1, U1> &&
         std::is_constructible_v<T2, U2>
explicit constexpr pair(U1&&, U2&& );
};

```

These aren't conceptually two constructors and they aren't meaningfully two constructors. The intent of the design is to have one single, conditionally explicit constructor. While this trick works, it's a fairly tedious, verbose, repetitive solution. How many third party libraries have code that *should* follow this idiom but doesn't due to the difficulty? Let's do better.

Proposal

The proposal is simply to allow for the direct declaration of conditionally `explicit` constructors (and conversion operators) in the same way that we currently specify that a function is conditionally `noexcept`: with an extra boolean constant argument. That would allow the above example to be written much more directly as:

SFINAE	SFINAE + explicit(bool)
<pre> template <typename T1, typename T2> struct pair { template <typename U1=T1, typename U2=T2, std::enable_if_t< std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> && std::is_convertible_v<U1, T1> && std::is_convertible_v<U2, T2> , int> = 0> constexpr pair(U1&&, U2&&); template <typename U1=T1, typename U2=T2, std::enable_if_t< std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> && !(std::is_convertible_v<U1, T1> && std::is_convertible_v<U2, T2>) , int> = 0> explicit constexpr pair(U1&&, U2&&); }; </pre>	<pre> template <typename T1, typename T2> struct pair { template <typename U1=T1, typename U2=T2, std::enable_if_t< std::is_constructible_v<T1, U1> && std::is_constructible_v<T2, U2> , int> = 0> explicit(!std::is_convertible_v<U1, T1> !std::is_convertible_v<U2, T2>) constexpr pair(U1&&, U2&&); }; </pre>
Concepts	Concepts + explicit(bool)

```

template <typename T1, typename T2>
struct pair {
    template <typename U1=T1, typename U2=T2>
        requires std::is_constructible_v<T1, U1> &&
            std::is_constructible_v<T2, U2> &&
            std::is_convertible_v<U1, T1> &&
            std::is_convertible_v<U2, T2>
        constexpr pair(U1&&, U2&& );

    template <typename U1=T1, typename U2=T2>
        requires std::is_constructible_v<T1, U1> &&
            std::is_constructible_v<T2, U2>
        explicit constexpr pair(U1&&, U2&& );
};

```

```

template <typename T1, typename T2>
struct pair {
    template <typename U1=T1, typename U2=T2>
        requires std::is_constructible_v<T1, U1> &&
            std::is_constructible_v<T2, U2>
        explicit(!std::is_convertible_v<U1, T1> ||
            !std::is_convertible_v<U2, T2>)
        constexpr pair(U1&&, U2&& );
};

```

In both cases, the constructor at the right is `explicit` if the condition in parentheses is `true`, otherwise it's not. One truly, conditionally explicit constructor - finally a direct way to express our intent.

This is a pure language extension, such syntax is ill-formed today, and no existing code will break.

Proposed Wording

In 10.1.2 [dcl.fct.spec] paragraph 1:

Function-specifiers can be used only in function declarations.

function-specifier:

`virtual`

`explicit`

explicit-specifier

explicit-specifier:

`explicit` (*constant-expression*).

`explicit`

In 10.1.2 [dcl.fct.spec] paragraph 3:

~~The `explicit` specifier~~An *explicit-specifier* shall be used only in the declaration of a constructor or conversion function within its class definition [...]

Insert new paragraph after 10.1.2 [dcl.fct.spec] paragraph 3:

In an *explicit-specifier*, the *constant-expression*, if supplied, shall be a contextually converted constant expression of type `bool`. If that constant expression is true, the function is explicit. Otherwise, the function is not explicit. A `(` token that follows `explicit` is part of the *explicit-specifier* and does not commence an initializer. The *explicit-specifier* `explicit` without a *constant-expression* is equivalent to the *explicit-specifier* `explicit(true)`.

In 11.3 [dcl.meaning], paragraph 2:

A `static`, `thread_local`, `extern`, `mutable`, `friend`, `inline`, `virtual`, `constexpr`, ~~`explicit`~~; or `typedef` specifier or `explicit-specifier` applies directly to each *declarator-id* in an *init-declarator-list* or *member-declarator-list*; [...]

In 11.6.1 [dcl.init.aggr], paragraph 1, change `explicit` to not use code font:

An *aggregate* is an array or a class with

- no user-provided, ~~`explicit`~~ `explicit`, or inherited constructors (15.1 [class.ctor]),
- [...]

In 15.1 [class.ctor], paragraph 1:

In a constructor declaration, each *decl-specifier* in the optional *decl-specifier-seq* shall be `friend`, `inline`, ~~`explicit`~~, or `constexpr`, or an `explicit-specifier`.

In 15.3.1 [class.conv.ctor], paragraph 1:

A constructor that is not `explicit` (10.1.2 [dcl.fct.spec]) ~~declared without the function-specifier `explicit`~~ specifies a conversion from the types of its parameters (if any) to the type of its class. Such a constructor is called a *converting constructor*.

In 16.3.1.7 [over.match.list], paragraph 1, change `explicit` to not use code font:

If the initializer list has no elements and `T` has a default constructor, the first phase is omitted. In copy-list-initialization, if an ~~`explicit`~~ `explicit` constructor is chosen, the initialization is ill-formed.

In 16.3.1.8 [over.match.class.deduct], paragraph 2, change `explicit` to not use code font:

Each such notional constructor is considered to be explicit if the function or function template was generated from a constructor or *deduction-guide* that was declared ~~`explicit`~~ `explicit`.

In 20.4.2.2 [functions.within.classes], the note is no longer necessary:

For the sake of exposition, the library clauses sometimes annotate constructors with `EXPLICIT`. Such a constructor is conditionally declared as either explicit or non-explicit (15.3.1 [class.conv.ctor]). ~~[Note: This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. —end note]~~

In 23.6.3.1 [optional.ctor], the note is no longer necessary:

~~[Note: The following constructors are conditionally specified as explicit. This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. —end note]~~

```
template<class U = T> EXPLICIT constexpr optional(U&& v);
```

Acknowledgements

Thanks to Titus Winters, whose EWG Wishlist thread led to the creation of this proposal.